AI Systems Performance Engineering

Chapter 6 — GPU Architecture, CUDA Programming, and Maximizing Occupancy

Tianqin Cai
Applied Scientist
AWS

Internal Training for Applied Scientists on AI Infrastructure

August 15, 2026

What This Chapter Covers

1. **SIMT execution model** — how **warps**, **thread blocks**, and **grids** map your algorithm onto **SMs**. (Part I)
2. **CUDA programming patterns** — kernel anatomy, launch parameters, asynchronous allocation. (Part II)
3. **Memory hierarchy** — register file, shared/L1, L2, HBM3e, plus asynchronous data movement (**TMA**, **TMEM**). (Part III)
4. **Roofline analysis** — is a kernel **compute-bound** or **memory-bound**? Push toward **peak throughput**. (Parts IV–V)

Where does occupancy fit?

Occupancy is the working metric that connects the SIMT model (Part I) to peak throughput (Parts IV–V): enough **resident warps** keep SMs busy while data moves.

Plain English: the arc: ***how the GPU executes*** *→ how you program it → where data lives →* ***what limits speed****.*

Roadmap: Five Parts, Each Answers One Question

Part	Slides	The question it answers
I GPU architecture	4–20	How does **SIMT** map warps/blocks/grids onto SMs?
II CUDA programming	21–28	How do we **map a computation onto GPU threads**?
III Memory hierarchy	29–37	Where does data **live**, and what does **movement cost**?
IV Occupancy in practice	38–46	**When** do more warps actually help?
V Correctness & roofline	47–50	Optimize **compute** next, or **memory**?

The whole chapter in one sentence

Expose enough parallelism, keep data close, and profile the real bottleneck.

When a detail page appears (SFU, TMEM, Unified Memory. . .), it always belongs to one of these five questions.

Part I — GPUs Optimize Throughput; CPUs Optimize Latency

- CPUs: few cores, deep caches, fast *single* threads. GPUs: hundreds of **SMs** (streaming multiprocessors) running **thousands of threads** in parallel.
- Basic CUDA flow (right): host copies data **CPU** $\rightarrow$ **GPU**, launches a **kernel**, copies results back.
- The GPU's edge is **massive parallelism**: many SMs process different data **at once**; within an SM, **resident warps** also fill memory stalls.
- Sweet spot: **data-parallel** work — matrix multiplies, convolutions — written in CUDA C++ or via **PyTorch** / OpenAI **Triton**.[1,3]

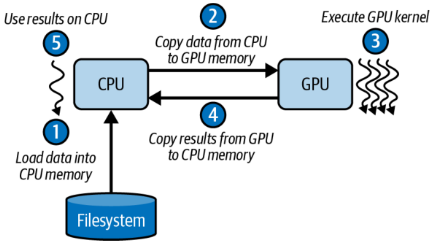


Figure 6-1. Simple CUDA programming flow

Plain English: the CPU finishes one part fast; the GPU is a **factory** *— keep* **every workshop busy***.*

Inside a Blackwell SM: The Resource Budget

- Each SM tracks up to **64 warps** (**warp** = 32 threads; details p. 12) = **2,048 threads** — the pool the scheduler switches between.
- **64K 32-bit registers** per SM (256 KB total); any single thread can use at most **255**.
- **256 KB** combined L1 cache / shared memory; up to **228 KB** configurable as user-managed shared memory (**227 KB usable** — CUDA reserves 1 KB per block).
- These on-chip budgets let one SM juggle thousands of threads without constantly touching DRAM.[2]

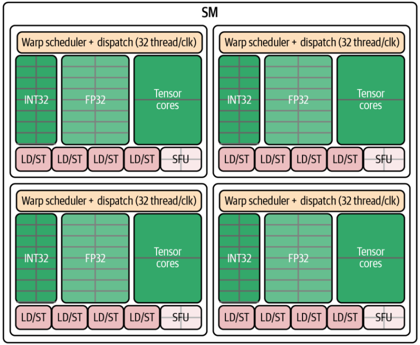


Figure 6-2. A Blackwell SM: four scheduling partitions sharing on-chip resources

Plain English: an SM is a workshop: registers are toolbelts, shared memory is the workbench.

Four Warp Schedulers, Dual Issue

- Each SM = four "**mini-SMs**": independent warp schedulers with their own dispatch logic.
- Each scheduler issues **1 warp instruction per cycle** ⇒ up to **4 warps advance per cycle**.
- **Dual-issue**: one math (INT32/FP32/Tensor Core) + one memory (load/store) instruction per cycle — *from the same warp*, never across warps.
- Best case: **4 math + 4 memory** instructions in flight every cycle.

Per cycle (Table 6-1)	Limit
Schedulers	4
Warps issued	4
Math / memory ops	4 / 4

Book note: table values are illustrative; real benchmarks in the GitHub repo[7]

Plain English: **four dispatchers** *on one floor, each sending out one crew per beat — and a crew can compute with one hand while fetching parts with the other.*

SFUs and Load/Store Pipelines

SFU — Special Function Unit

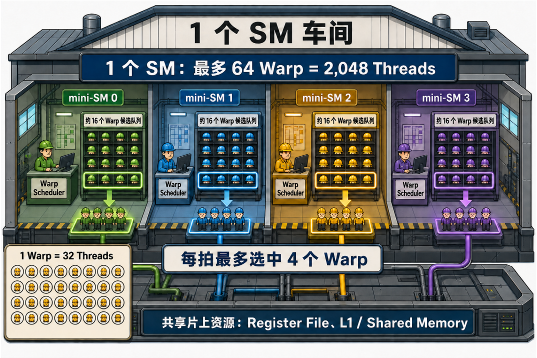
- Handles **transcendentals**: sine, cosine, reciprocal, square root.
- Own **dedicated pipeline** — *not* part of the dual-issue math+memory pair.
- Slow functions never stall the core compute/memory pipes ⇒ more **instruction-level parallelism** for mixed-operation kernels.

LD/ST — Load/Store pipelines

- **16 LD/ST pipelines** per SM (4 per scheduler) feed L1/shared, L2, and global memory.
- **Caveat (book warning):** exact counts and pairings are **not guaranteed** — rely on profiling counters and the **Blackwell tuning guide**.[2]

Plain English: the SFU is a specialist counter at the back of the store — complicated requests go there so the express lanes keep moving.

Analogy: One SM Is a Workshop



Presenter's analogy figure (not from the book)

- Four bays = **4 scheduling partitions** ("mini-SMs"), one **warp scheduler** each — CUDA only ever sees the whole SM.
- Hard hats on shelves = **resident warps**: ∼16 per partition, **64 total**. Resident = state already allocated — *not* "executing right now."
- Highlighted crews = the **≤4 warps issued this cycle** (each scheduler picks at most 1 *ready* warp).
- Bottom left: **1 warp = 32 threads** — dispatched as a whole crew.
- Common foundation: **register file, L1, shared memory** — shared by all four partitions.

Plain English: 64 crews on standby, 4 dispatchers, at most 4 crews per beat — each crew is 32 workers.

The Thread Hierarchy: Threads → Blocks → Grids

- **Thread**: runs your kernel code on one data element.
- **Thread block** (CTA — cooperative thread array): up to **1,024 threads**; shares fast on-chip **shared memory**.
- **Grid**: all blocks of one launch — sized right, it scales to **millions of threads** with zero kernel changes.
- The CUDA runtime (and PyTorch) handles scheduling and distribution across all SMs.

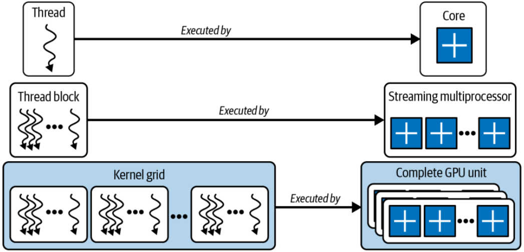


Figure 6-3. Threads, thread blocks (CTAs), and grids

Plain English: workers (threads) form teams (blocks) that talk cheaply inside the team; the company (grid) hires as many teams as the job needs.

Blocks Cooperate Inside, Stay Independent Outside

- Within a block: share data via shared memory, synchronize with `__syncthreads()` (a barrier: everyone arrives before anyone continues).
- **Every barrier costs time** — the book's advice: **minimize synchronization points**.
- Across blocks: **independent, no guaranteed order**. That freedom lets the scheduler spray blocks over all SMs — and lets your kernel run **unmodified on future GPUs** with more SMs.

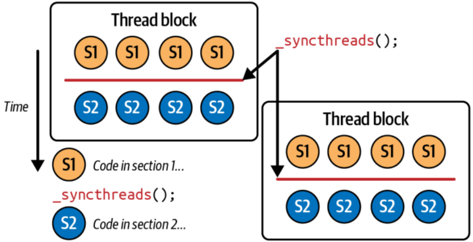


Figure 6-6. `__syncthreads()` between two code sections

Plain English: team huddles (barriers) are useful but expensive — call as few as possible; and never assume team A finishes before team B.

Thread Block Clusters and DSMEM

- Traditionally, threads in *different* blocks could not cooperate. Modern GPUs add **thread block clusters**: blocks that communicate **across SMs**, with cluster-scoped hardware barriers.
- Backed by **DSMEM** (distributed shared memory): the shared-memory banks of participating SMs, linked over a fast **on-chip interconnect** into one pool.
- Threads read, write, and atomically update each other's shared buffers **at on-chip speeds — without global-memory bandwidth**.
- Key enabler for huge matmuls in LLMs — details in **Chapter 10**.

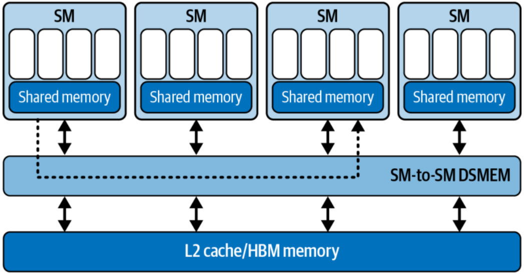


Figure 6-5. DSMEM shared across thread blocks in a cluster

Plain English: neighboring teams knock a door through the wall between their workbenches instead of mailing parts through the warehouse.

Warps and SIMT: 32 Threads in Lockstep

- Blocks are subdivided into **warps of 32 threads** that execute one instruction together under **SIMT** (single instruction, multiple threads).
- The warp — not the thread — is what the **hardware actually schedules**; warp size has been **32 on every GPU generation**.
- The hardware hides long-latency events (global loads, cache fills, pipeline stalls) by **rapidly switching among warps**.

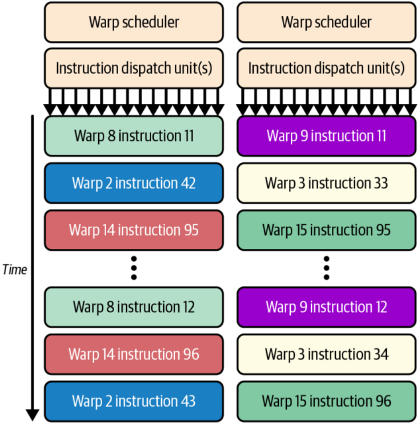


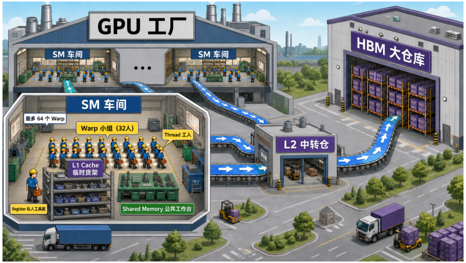
Figure 6-7. A warp advances as a whole under the warp scheduler

Plain English: a warp is a **32-worker crew***: one instruction for all 32, everyone moves together — or the whole crew waits.*

Two Views of One Machine: Software vs. Hardware

Software (you write)	Hardware (runs it)	Analogy
Grid	**all SMs**	the batch of orders
Block	**one SM** (for life)	one work team
(warp: implicit)	the issue unit	32-worker crew
Thread	one lane of a warp	one worker

- You choose **grid and block sizes**; hardware decides placement and scheduling. A block **never migrates** — but one SM usually **hosts several blocks**.
- The **warp is the bridge**: hardware slices every block into warps of 32 — automatically, invisibly.
- Memory preview (Part III): registers = toolbox, shared = workbench, L1 = shop shelf, **L2 = central depot**, **HBM = warehouse**; a global load tries L1 → L2 → HBM and returns the same way.



Presenter's analogy: SM workshops, L2 depot, HBM warehouse

Plain English: software speaks in **grids and blocks***; hardware answers in* **SMs and warps** *— the warp is where they meet.*

Worked Example: One Million Adds Through Both Views

Software view: you write the order sheet

```
add<<<3907, 256>>>(A, B, C, N);
```

Grid	1 (this launch)
Blocks	**3,907** $= \lceil 10^6/256 \rceil$
Threads / block	**256** (your choice)
Warps / block	**8** ($= 256 \div 32$, HW slices)
Total	$\sim$1M threads, 31,256 warps

Each thread only says: "I handle element `idx`."

Hardware view: toy GPU — 4 SMs, 2 blocks/SM

1. First **8 blocks** move in: 2 per SM.
2. Each SM now holds $2 \times 8 =$ **16 resident warps**.
3. Schedulers keep issuing whichever warps are **ready** (= latency hiding — next page).
4. A block finishes $\Rightarrow$ the next of the **3,899 waiting** blocks moves in — batch by batch until all 3,907 are done.

- 4 SMs or 400 SMs: **same code** — the grid **decouples** the program from the GPU's size.

Plain English: you write the **order sheet** *(3,907 teams of 256 workers); the hardware assigns teams to workshops and slices each team into 32-worker crews.*

Let the Workshop Move: Latency Hiding, Cycle by Cycle

One partition, one moment

Warp A — **waiting** on an HBM load
Warp B — **waiting** on a previous result
Warp C — **ready**   Warp D — **ready**

The scheduler skips A and B and **issues C or D** — skipping a stalled warp costs **nothing**.

Cycle	Issued (example)
1	Warp C
2	Warp F
3	Warp D
4	Warp H

- "Issued" = the instruction **starts**; it need not finish this cycle.
- Latency hiding does **not** shorten any warp's wait — it **fills the wait** with other warps' work.
- This is why **64 resident warps** matter: a deep pool of *ready* candidates for the 4 dispatchers — and why low occupancy hurts (nothing ready to switch to).

Plain English: latency hiding does ***not*** *reduce waiting time — it* ***fills the waiting time*** *with other crews' work.*

Occupancy Gives the Scheduler More Choices

Definition

Occupancy = active warps on an SM ÷ the hardware maximum (64 on Blackwell).
"**Achieved occupancy**" is the measured average, reported by Nsight Compute.

- High occupancy ⇒ when one warp stalls on memory, another is ready ⇒ **latency hiding**.
- Blackwell's large register file (64K/SM) makes high occupancy easier to reach.
- The counterweight: warps share **registers and shared memory**. Pack in too many ⇒ **register spilling** to slow memory — a new stall you created.
- So: profile occupancy **together with** register / shared-memory usage.

Plain English: occupancy is **not** *"how busy the GPU is" and* **not** *a performance score — it counts* **how many resident crews the dispatcher can choose from**. *Enough hides latency; beyond enough, more may not help* *(p. 52)**.*

Warp Divergence: When `if/else` Splits the Crew

- If threads *within one warp* take different branches, the warp **serializes**: it runs the `if` path with the other lanes **masked**, then the `else` path.
- Execution time multiplies by the **number of distinct paths**.
- **No penalty across warps** — different warps may branch differently for free.
- Practical corollary: branch on **warp-aligned** conditions (`threadIdx.x / 32`), not per-thread parity (`threadIdx.x % 2`).
- Detection and mitigation: Chapter 8 (Nsight Compute).

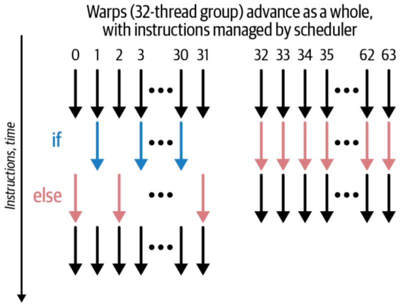


Figure 6-8. SIMT warp divergence (left) vs. uniformity (right)

Plain English: if half the **crew** *takes one work order and half another, the crew runs* **each job twice***.*

Hardware Limits I: Warp and Block (Table 6-2)

No need to memorize these numbers. The one idea: each block's threads, registers, and shared memory decide how many blocks and warps still fit on one SM.

Resource	**Limit (B200)**
Warp size	32 threads
Max threads / block	1,024
Max warps / block	32 $(= 1024 \div 32)$

`blockDim.x` $\times$ `blockDim.y` $\times$ `blockDim.z` $\leq$ 1024

Why multiples of 32 matter

A **33-thread block** occupies **two** warp slots — the second warp runs **1 of 32 lanes** but still consumes a full scheduler slot. Every non-multiple of 32 donates scheduler capacity to the void.

- Block too big $\Rightarrow$ too many registers per block $\Rightarrow$ **spilling**; or too much shared memory (227 KB usable is shared by **all resident blocks** on the SM).
- Smaller blocks can mean **higher occupancy** — more independent blocks fit per SM.

Hardware Limits II: SM-Resident and Grid (Tables 6-3 / 6-4)

Per SM (resident)	**Limit**
Max warps	**64** (= 2,048 threads)
Max threads	2,048
Max blocks	32
Grid	
Max blocks (X / Y / Z)	∼2.1B / 65,535 / 65,535
Max concurrent grids	**128** kernels per device

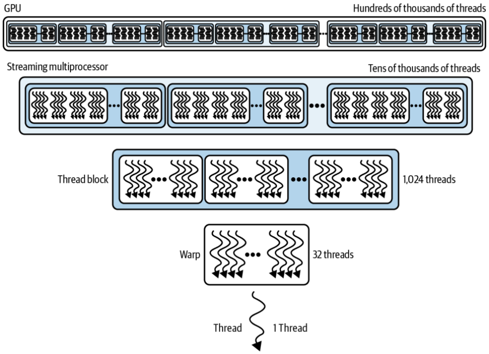


Figure 6-9. Relative scale of thread resources

- Worked example: 1024-thread blocks ⇒ only **2 blocks/SM**; 256-thread blocks ⇒ **8 blocks/SM** — same 2,048 threads, more flexibility.
- In practice you hit **per-SM limits** long before grid limits; >65,535 blocks in Y/Z ⇒ use 2D/3D grids or multilaunch.[1,2]

Compatibility: PTX, SASS, and Fatbins

- **SASS**: final machine code for *one* architecture (sm_90 = Hopper, sm_100 = Blackwell). SASS-only binaries **won't run on newer GPUs**.
- **PTX**: virtual ISA the driver **JIT-compiles** at load time ⇒ **forward compatibility**.
- Family targets (`sm_100f`) restrict portability to the same feature family.

Best practice

Ship a **fatbin**: generic PTX + the family-specific cubins you need for optimizations — with **fallback paths** for other architectures.

- Verify with `CUDA_FORCE_PTX_JIT=1`: forces PTX JIT; if the binary lacks PTX, **kernel launch fails** — rebuild with PTX.

Plain English: fast code must also **survive GPU generations** *— ship optimized* **SASS** *for today's chips plus* **PTX** *for forward compatibility.*

Part II — Anatomy of a CUDA Kernel: Device Side

The hardware wants many ready warps — now: how does CUDA code create them?

```
__global__ void myKernel(float* input, int N) {
    // unique global index for this thread
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < N) { // bounds check!
        input[idx] *= 2.0f; // in-place: double it
    }
}
```

- `__global__`: runs on the device, callable from the host.
- Built-ins: `blockIdx` (which block am I), `blockDim` (block size), `threadIdx` (my rank in the block).

- The kernel is written from the perspective of **one thread**; `idx` carves out its slice of the data.
- CUDA compiles it into device code executed by **thousands or millions** of lightweight threads in parallel.
- Launch syntax (host): `<<<blocksPerGrid, threadsPerBlock>>>` — the two core parameters of *every* kernel invocation.

Plain English: write the job description for one worker; CUDA prints a million copies, each with a different row number.

Host Side: The Six-Step Data Flow

```
const int N = 1'000'000;
float *h_input = nullptr, *d_input = nullptr; // h_=host, d_=device
cudaMallocHost(&h_input, N * sizeof(float)); // 1) pinned host alloc
for (int i = 0; i < N; ++i) h_input[i] = 1.0f; // init
cudaMalloc(&d_input, N * sizeof(float)); // device alloc
cudaMemcpy(d_input, h_input, N * sizeof(float),
          cudaMemcpyHostToDevice); // 2) H2D copy
const int threadsPerBlock = 256;
const int blocksPerGrid = // = 3,907 for 1M
    (N + threadsPerBlock - 1) / threadsPerBlock;
myKernel<<<blocksPerGrid, threadsPerBlock>>>(d_input, N); // 3) launch
cudaDeviceSynchronize(); // 4) wait for device
cudaMemcpy(h_input, d_input, N * sizeof(float),
          cudaMemcpyDeviceToHost); // 5) D2H copy
cudaFree(d_input); cudaFreeHost(h_input); // 6) cleanup
```

- Naming convention: `h_` = host pointer, `d_` = device pointer — used throughout the book.
- `cudaMallocHost` = **pinned** (page-locked) memory — prerequisite for async copies to truly overlap later.
- Book note: *not fully optimized yet* — this is the simple, complete **template** the rest of the book improves on.

Why Pass N? The Single-Thread Perspective

The book's answer

"A CUDA kernel function is designed to work **inside of a single thread**, alongside thousands of other threads, **on a partition of the input data**. As such, N defines the size of the partition that this particular kernel will process."

- A CPU function could just inspect the array size. A kernel **cannot** — it sees a raw pointer and must be told where its world ends.
- Combined with `blockIdx`/`blockDim`/`threadIdx`, N lets every element be processed **cleanly and uniquely**, in parallel, across many SMs.
- This is the core mental shift from CPU to GPU programming: you don't write "process the array," you write "process **my** element — if it exists."

Plain English: each of the million copied workers gets the same memo; N is the line that says where the job ends.

The Bounds Check — and Lazily Surfacing Errors

Worked example: $N = 63$

Scheduler assigns **2 warps** (64 threads). Warp 1 handles elements 0–31 — fine. Warp 2 expects 32–63 **plus one extra thread**; without `if (idx < N)` it reads past the array ⇒ `cudaErrorIllegalAddress`.

- Book rule: bounds checks are everywhere in CUDA; "if you don't see it, **understand why it's not there**."

- Kernels run **asynchronously, with no per-thread exceptions**: an illegal access sets a **global fault flag** for the whole launch.
- The host only sees it at the **next sync or CUDA API call** — errors surface **lazily**.
- Practice: `cudaGetLastError()` + `cudaDeviceSynchronize()` right after launches, so faults are caught where they happen.

Plain English: the GPU doesn't call you when something breaks — it leaves a note you only find the next time you check the mailbox.

Choosing Launch Parameters: The 256 Recipe

Recipe that works almost everywhere

Start with `threadsPerBlock = 256` (8 warps); `blocksPerGrid = (N + 255) / 256` (round up — every element covered). Tune 128/512 by profiling.

- **Multiple of 32**: no half-empty warps occupying scheduler slots.
- **Latency hiding**: hundreds of threads per SM cover DRAM and instruction stalls — 8 blocks $\times$ 256 $=$ 2,048 *can* fill an SM, **if registers and shared memory allow**.
- **Occupancy**: 8 warps/block usually fits registers and shared memory comfortably.
- **Resource-balanced**: far from the 1,024 cap, big enough to keep warps busy.
- Book tip for Blackwell: consider **256–512** threads per block.

Plain English: 256 is the "medium coffee" of block sizes — almost never the wrong first order, refine later if profiling says so.

2D and 3D Kernel Inputs

```
__global__ void my2DKernel(float* input,
                           int width, int height) {
    int x = blockIdx.x * blockDim.x + threadIdx.x;
    int y = blockIdx.y * blockDim.y + threadIdx.y;
    if (x < width && y < height) { // 2D bounds check
        int idx = y * width + x; // flatten to 1D
        input[idx] *= 2.0f;
    }
}
// host: dim3 threads(16,16); // 256 threads, 2D
// dim3 blocks((W+15)/16, (H+15)/16);
```

- Natural fit for **images**: e.g., a 1,024 × 1,024 matrix processed by 16 × 16 blocks (still 256 threads).
- Same pattern generalizes to 3D with `dim3(x, y, z)` — volumetric data maps directly onto the thread hierarchy.
- Book note: most of the book uses **1D or 2D (tiled)** launches; in 1D, plain ints beat `dim3`.

Plain English: same recipe, two axes: each worker gets a (row, column) badge instead of a single row number.

Allocate Asynchronously: Streams and Memory Pools

```
cudaStream_t s;
cudaStreamCreateWithFlags(&s, cudaStreamNonBlocking);

float* d_buf = nullptr;
cudaMallocAsync(&d_buf, N * sizeof(float), s);
myKernel<<<blocks, threads, 0, s>>>(d_buf, N);
cudaFreeAsync(d_buf, s); // deferred until s finishes
```

- **Free waits only for stream `s`** — no global `cudaDeviceSynchronize`, no cross-stream sync.

- `cudaMalloc`/`cudaFree` are **synchronous and expensive**: full-device sync + OS calls (`mmap`/`ioctl`), kernel-space context switches.
- `cudaMallocAsync`/`cudaFreeAsync` draw from a per-device **memory pool**: recycled buffers, no OS round trips, **less fragmentation** over long training loops.
- Non-blocking streams avoid legacy **default-stream barriers** (book tip; multistream overlap in Chapter 11).

Plain English: keep a truck fleet in the lot and grab keys — don't buy and return a truck for every delivery.

Tuning the Pool; PyTorch's Caching Allocator

Pool knobs

- `cudaMemPoolAttrReleaseThreshold` (via `cudaMemPoolSetAttribute`): how much reserved memory the pool keeps before releasing to the OS.
- `cudaMemPoolTrimTo`: proactively return memory.
- Trade-off: total **footprint** vs. **fragmentation**.

The PyTorch connection

- PyTorch's **caching allocator** (`PYTORCH_ALLOC_CONF`, formerly `PYTORCH_CUDA_ALLOC_CONF`) is the same idea: reuse GPU memory, avoid a synchronous `cudaMalloc` per new tensor per iteration.

- Verdict from the book: one-off buffers — blocking `cudaMalloc` is fine; **allocation-heavy loops — async + pools** give more consistent performance and higher throughput.

Part III — The Memory Ladder at a Glance

We can now create plenty of threads — but what are they all waiting for? Usually data.

Level	Scope	Capacity	Latency	BW
Registers	thread	256 KB / SM	$\sim$1 cyc	10s TB/s
Shared + L1	SM	228 KB usable	20–30 cyc	TB/s
TMEM	SM	256 KB (TC)	$\sim$10 cyc	TB/s
Const cache	SM	8 of 64 KB	1 cyc bcast	TB/s
L2 cache	GPU	**126 MB**	$\sim$200 cyc	multi-TB/s
Local (spill)	thread	DRAM-backed	$\leq$1000 cyc	—
Global HBM3e	device	**180 GB** (B200)	$\leq$1000 cyc	**$\sim$8 TB/s**

Table 6-5. Every level trades capacity for latency/bandwidth.

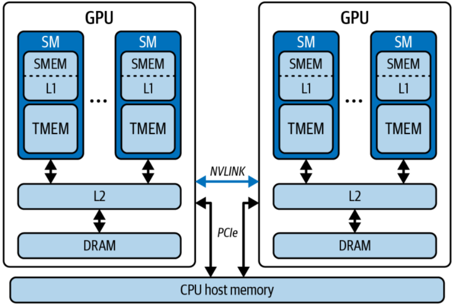


Figure 6-10. GPU memory hierarchy, including the CPU

Plain English: your **own toolbox** *(registers) → the* **shared workbench** *(shared) → the factory's* **central depot** *(L2) → the* **distant warehouse** *(HBM) — do the work at your bench.*

Register Pressure Can Turn Fast Storage into DRAM Access

- Every thread "begins its journey" at the **register file**: single-cycle reads/writes, contending with almost nothing — **tens of TB/s per SM**.
- Budget: 64K registers per SM, **max 255 per thread**.
- The cliff: need more (too many locals, compiler temporaries) $\Rightarrow$ overflow **spills into local memory** — which despite the name lives in **off-chip DRAM**: hundreds to 1000+ cycles.
- Watch **"Registers Per Thread"** in Nsight Compute — spills are silent killers.

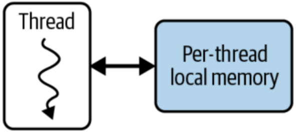


Figure 6-12. Per-thread local memory (spill space, DRAM-backed)

Plain English: "local memory" is a self-storage unit across town, not your pocket.

Shared Memory + L1: One SRAM, Two Jobs

- One **256 KB SRAM per SM**, split between user-managed **shared memory** (up to 228 KB; 227 usable per block) and **L1/data cache**.
- You choose the **carveout**:

```
cudaFuncSetAttribute(myKernel,
  cudaFuncAttributePreferredSharedMemoryCarveout,
  carveoutPercent);
```

- ~20–30 cycles; **TB/s** if you avoid **bank conflicts** (multiple threads hitting the same memory bank ⇒ serialized access).
- This is the workbench for **intrablock cooperation** — tiles, reductions, staging.

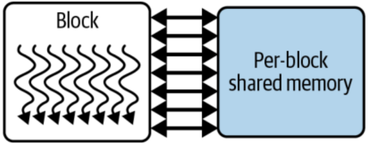


Figure 6-13. Thread-block shared memory

Plain English: one workbench, adjustable split: how much is your team's project table vs. the automatic cache shelf.

TMEM and TMA: Feeding the Tensor Cores

- **TMEM**: dedicated **256 KB per-SM SRAM**, the **accumulator** for 5th-gen Tensor Core ops (`tcgen05.*`, **UMMA**); talks to Tensor Cores at tens of TB/s.
- **Not pointer-addressable** from CUDA C++ — data movement is orchestrated by the **TMA** (Tensor Memory Accelerator) via **descriptors**.
- For $C = A \times B$: operand B in SMEM, A + accumulator in TMEM; TMA moves tiles HBM $\to$ L2 $\to$ SMEM; SMEM $\leftrightarrow$ TMEM implicit via Tensor Core ops.
- Net effect: **reduces Tensor Core reliance on global memory**. Details: Chapter 10.

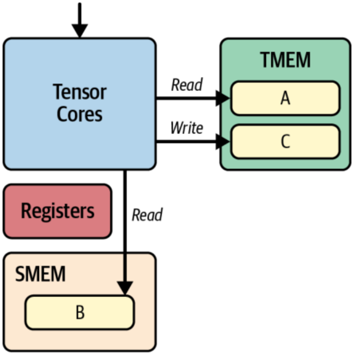


Figure 6-11. TMEM + SMEM servicing Tensor Cores for $C = A \times B$

Plain English: TMA is a forklift crew moving pallets in the background; the chefs never leave the kitchen.

Constant Cache: One-Cycle Broadcast

- $\sim$**8 KB cache per SM** fronting the 64 KB `__constant__` space.
- Superpower: when all 32 threads of a warp load **the same address**, the cache **broadcasts** the value in a **single cycle** — as fast as a register.
- Divergent reads **serialize across lanes**; misses cost more — so it's for **small, read-only, uniformly-accessed** data.

The book's LLM examples

- Rotary positional encoding (RoPE) lookup tables
- ALiBi slopes
- LayerNorm γ/β vectors
- Embedding quantization scales

Shared by every thread with **zero global-memory traffic**.

Plain English: a megaphone, not a mailbox: one announcement reaches all 32 workers at once — but only if they all asked the same question.

L2 Cache: The GPU-Wide Middleman

- **126 MB** shared by all SMs — the glue between on-chip SRAM and off-chip HBM3e.
- $\sim$200 cycles, aggregate bandwidth in the **tens of TB/s**; absorbs L1 spillover.
- **Cross-block reuse**: data fetched by one thread block is reused by others **without revisiting DRAM**.

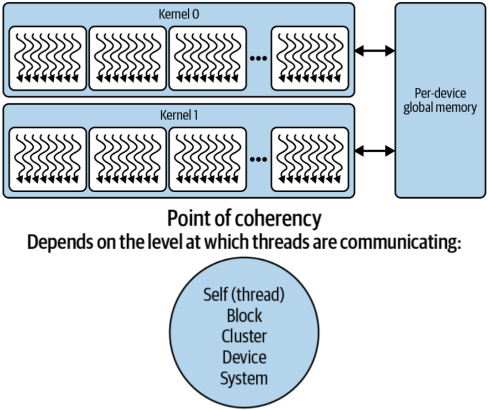
The coalescing rule (book tip)

Structure global loads into **128-byte aligned, coalesced** segments that map cleanly to cache lines — avoids split transactions, maximizes L2 **and** DRAM bandwidth. (How-to: Chapter 7.)

Plain English: L2 is the factory's **central depot***: if another crew already fetched the part from the warehouse, you grab it here — provided everyone orders* **whole pallets***, not single screws.*

Global HBM3e, the Dual-Die B200, and Coherency

- **180 GB** on B200 (∼288 GB on B300) at **∼8 TB/s** — but **hundreds to 1000+ cycles**: the slowest link in the chain.
- Register spills and oversized automatic arrays (local memory) also pay this price.
- **Fun fact (book note)**: B200 = **two reticle-limited dies** joined by a **10 TB/s** chip-to-chip interconnect, 4 HBM3e stacks each — presented to you as **one GPU, one address space**.
- **Point of coherency** (Figure 6-15): memory consistency is established at thread → block → cluster → device → system scope — the wider the communication, the higher the cost.



Figures 6-14 / 6-15. Global memory; points of coherency

Unified Memory Hides Copies, Not Their Cost

- `cudaMallocManaged()`: one coherent CPU+GPU address space — no separate buffers, no manual `cudaMemcpy`.
- Under the hood: pages **migrate on demand** over whatever link connects CPU and GPU.
- The catch: a GPU touch of a CPU-resident page **page-faults and stalls the kernel**.
- On PCIe: on-fault transfers can be **slower than manual memcpy**. On Grace superchips, **NVLink-C2C** (~900 GB/s) makes migrations near device-native — **but latency is never zero**.

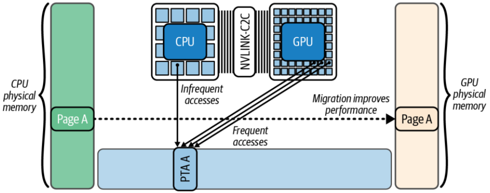


Figure 6-16. Automatic page migration with Unified Memory

Plain English: Unified Memory is ***automatic parts delivery****: convenient — but if the part isn't in the workshop when the crew needs it,* ***the crew stands and waits****.*

Taming Unified Memory: Prefetch, Advise, Attach

```
// 1) move data BEFORE the kernel needs it:
cudaMemPrefetchAsync(ptr, size, gpuId, s);
// 2) tell the driver how you'll use it:
cudaMemAdvise(ptr, size,
    cudaMemAdviseSetPreferredLocation, gpuId);
cudaMemAdvise(ptr, size,
    cudaMemAdviseSetReadMostly, gpuId);
cudaMemAdvise(ptr, size,
    cudaMemAdviseSetAccessedBy, otherGpuId);
// 3) confine faults/migrations to one stream:
cudaStreamAttachMemAsync(stream, ptr, 0,
    cudaMemAttachSingle);
```

- **Prefetch** turns first-touch migrations into **overlappable async transfers** (Figure 6-17).
- **ReadMostly** allows replicated read-only copies; **AccessedBy** lets a second GPU map pages **without** migration (Figure 6-18).
- **Attach** stops *other* streams from stalling on this range.
- No NVLink-C2C? Pin data **NUMA-local** (peer copy / prefetch).
- Result: performance **close to manual** `cudaMemcpy`, simplicity retained.

Part IV — Occupancy Ground Rules

The memory ladder explains why warps stall. Occupancy asks: how many other warps do we need to fill those gaps?

The most fundamental rule of CUDA performance (book)

"Launch enough parallel work to fully occupy the GPU."

Rule 1 — low occupancy, poor perf

First remedy: **increase parallelism** (more blocks/threads) until there are **enough ready warps to hide latency** — stop when performance **plateaus**.

Rule 2 — occupancy already high

If the kernel is **memory-bound**, pushing to 100% won't help: you need **just enough warps to hide latency** — beyond that, the bottleneck is bandwidth.

- Up next: one operation ($C = A + B$, $N = 10^6$), two implementations, and what the profilers say about them.

Plain English: rule 1 **fills the workshop with crews***; rule 2: a full workshop* **still stalls** *if the warehouse truck is the bottleneck.*

Case Study, Take 1: addSequential

```
// Single thread does ALL N additions
__global__ void addSequential(const float* A,
    const float* B, float* C, int N) {
  if (blockIdx.x == 0 && threadIdx.x == 0) {
    for (int i = 0; i < N; ++i) {
      C[i] = A[i] + B[i];
    }
  }
}
// Note: this kernel assumes <<<1,1>>>.
addSequential<<<1,1>>>(d_A, d_B, d_C, N);
cudaDeviceSynchronize();
```

- One thread, one warp, one SM — **everything else watches**.
- Book: "the GPU's vast resources are mostly idle... very poor occupancy and, ultimately, low performance."
- No other warps to switch to ⇒ memory waits become **pure idle time** — zero latency hiding.

Plain English: you rented the whole factory and hired one worker.

The Same Trap in PyTorch

```python
import torch
N = 1_000_000
A = torch.arange(N, dtype=torch.float32,
                 device='cuda')
B = 2 * A
C = torch.empty_like(A)
torch.cuda.synchronize()
# Naive, sequential GPU ops - DO NOT DO THIS
with torch.inference_mode():
    for i in range(N): # N tiny serial kernels!
        C[i] = A[i] + B[i]
torch.cuda.synchronize()
```

- A Python loop around GPU ops **serializes N tiny kernel launches** — the GPU becomes "a scalar, nonparallel processor."
- Same terrible occupancy, plus **per-launch CPU overhead** × 1,000,000.
- Book advice: there is **almost always an optimized PyTorch-native (vectorized) implementation** — including compiler-emitted code.

Plain English: the most common GPU bug: accidentally using the GPU as a very expensive single-core CPU.

Case Study, Take 2: addParallel — and `C = A + B`

```
// One thread per element
__global__ void addParallel(
    const float* __restrict__ A,
    const float* __restrict__ B,
    float* __restrict__ C, int N) {
  int idx = blockIdx.x*blockDim.x + threadIdx.x;
  if (idx < N) C[idx] = A[idx] + B[idx];
}
addParallel<<<(N+255)/256, 256>>>(dA, dB, dC, N);
```

The book's full version also uses pinned host memory, a non-blocking stream, and `cudaMallocAsync`/`cudaMemcpyAsync` — the habits from Part II.

- ∼3,907 blocks × 256 threads — **a million threads** cover the array (Figure 6-19).
- `__restrict__`: promises no pointer aliasing — frees the compiler to optimize.
- PyTorch equivalent is one line: `C = A + B` — a single vectorized kernel engaging many threads concurrently.

Plain English: same factory, now every workstation is staffed — and the PyTorch version is just "hire the staffing agency."

Measuring It: nsys and ncu

```
# Nsight Systems: whole-app timeline
nsys profile --stats=true -t cuda,nvtx \
    -o report ./add_parallel.py

# Nsight Compute: per-kernel counters
ncu --section SpeedOfLight \
   --metrics sm__warps_active.avg.\
pct_of_peak_sustained_active,gpu__time_duration.avg \
   --target-processes all \
   --print-summary per-gpu -o report ./add_parallel.py
```

- **nsys** answers "**where does time go** — is the GPU starved or blocked?"
- **ncu** answers "**why is this kernel slow** — occupancy? stalls? cache misses?"
- Run **both**: nsys alone misses in-kernel inefficiency; ncu alone can't see whether kernels are **fed data fast enough**.
- That long `sm__warps_active...` metric **is** achieved occupancy.

Plain English: nsys is the stopwatch for the whole machine; ncu is the microscope on one kernel.

Parallelism Cuts Runtime 22× at Only 38.7% Occupancy

Metric	Sequential	Parallel
Kernel time (ms)	48.21	**2.17**
GPU utilization	1.5%	**95%**
Achieved occupancy	1.3%	**38.7%**
Warp exec. efficiency	3.1%	**100%**

Table 6-6 (illustrative — real benchmarks in the book's repo[7]). Warp efficiency 3.1% ≈ 1/32: one active lane per warp.

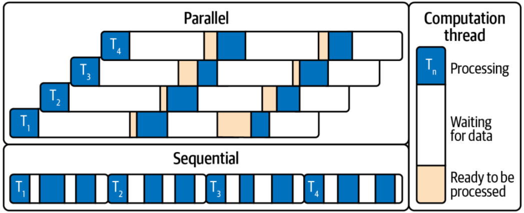


Figure 6-20. Sequential timeline has idle gaps; in the parallel version other warps' work fills them

- Note: 38.7% occupancy already bought 22× — you don't need 100%.
- GPU utilization, SM active %, achieved occupancy, and warp efficiency are **related but different metrics** — which is exactly how **95%**, **38.7%**, and **100%** coexist on one kernel.

A Busy GPU Can Still Be Waiting on Memory (LLM Decode)

95% utilization at only 38.7% occupancy, already 22× faster — so we don't need 100%. But are we at peak FLOPS? No: vector add mostly moves data.

- After parallelism, the next lever is per-warp efficiency (ILP etc., Chapter 8). But: **even at 100% occupancy**, performance suffers if the kernel is **memory bound** — limited by data movement, not compute.
- The book's canonical example: the **decode phase of an LLM** — every generated token streams **model weights** from HBM into registers/shared memory.
- Hundreds of billions of parameters × ∼1 byte ≈ **hundreds of GB** per pass — this **saturates memory bandwidth** regardless of thread count.

The trend that makes this worse (book note)

GPU FLOPS are outpacing memory bandwidth. Blackwell HBM3e delivers ∼8 TB/s, but compute and model sizes grow faster — **optimizing memory movement is absolutely critical** in modern AI workloads.

Plain English: when the whole job is "carry boxes from the warehouse," hiring more clerks at the desk doesn't help — that's what the roofline model (Part V) formalizes.

__launch_bounds__: Compile-Time Occupancy Control

```
// promise: never launched with > 256 thr/block
// request: keep >= 16 blocks resident per SM
__global__ __launch_bounds__(256, 16)
void myKernel(...) { /* ... */ }
```

- $16 \times 256 = 4{,}096 > 2{,}048$ limit $\Rightarrow$ compiler **clamps** to 8 blocks and warns:
  `ptxas warning: ... .minnctapersm will be ignored`

- Effect: compiler **caps per-thread registers** (and may restrict unrolling/inlining) to avoid spills and fit **more warps per SM**.
- The deal: trade a bit of **per-thread performance** for **more consistent warp throughput** (more warps in flight).
- Danger zone: forcing too many threads $\Rightarrow$ registers run out $\Rightarrow$ **spilling to local memory** — slower than what you started with.

Plain English: you're trading each worker's toolbox size for the number of workers on the floor — the sweet spot is empirical.

The Occupancy API: Runtime Autotuning

```
int minGridSize = 0, bestBlockSize = 0;
// if kernel uses extern __shared__, set truthfully:
size_t dynSmemBytes = 0;
cudaOccupancyMaxPotentialBlockSize(
    &minGridSize, &bestBlockSize, myKernel,
    dynSmemBytes, /*blockSizeLimit=*/0);
// cover N, but don't go below the min grid
// that saturates occupancy:
int gridSize = std::max(minGridSize,
    (N + bestBlockSize - 1) / bestBlockSize);
myKernel<<<gridSize, bestBlockSize,
           dynSmemBytes>>>(...);
```

- Computes the block size that **maximizes occupancy** given the kernel's actual register/shared-memory usage.
- Two gotchas (book): `minGridSize` saturates occupancy — it is **not** the grid that covers N (take the `max`); and **pass real dynamic shared-memory bytes**.
- Book tip: **validate with $\pm$1–2 candidate block sizes** — register pressure and L2 behavior can make slightly **sub-maximal occupancy faster**.

Part V — Debugging Correctness: NVIDIA Compute Sanitizer

No more blind-tuning block sizes — next: compute ceiling or memory ceiling? (roofline, after a correctness detour)

Memory

- **memcheck** — out-of-bounds / misaligned accesses (global, local, shared), HW exceptions, device leaks; `--check-device-heap`.
- **initcheck** — reads of uninitialized global memory (classic cause: a **missing H2D copy**).

Concurrency

- **racecheck** — shared-memory hazards: **WAW / WAR / RAW** ⇒ nondeterministic behavior.
- **synccheck** — invalid sync primitives, mismatched barriers ⇒ deadlocks, inconsistent state.

- CLI: `compute-sanitizer [--tool <name>] <app>`; filter with `--kernel-name`, annotate with **NVTX** (`--nvtx yes`).
- Book tip: run it in **CI** with `--error-exitcode`.[5]

Plain English: "it works on my run" means nothing at 100,000 threads — the sanitizer is your seatbelt.

Roofline Tells Us Which Resource Is the Limit

- **Arithmetic intensity** = FLOPs per byte moved to/from HBM.
- Two ceilings: **flat** compute roof (~80 TFLOP/s FP32) and **sloped** memory roof (~8 TB/s), crossing at the **ridge point** ≈ **10 FLOPs/byte** (80T ÷ 8T).
- Worked example: $C = A + B$ loads 8 B, adds once, stores 4 B ⇒ 1 FLOP / 12 B = **0.083 FLOPs/byte** — >100× left of the ridge: hopelessly **memory-bound**.
- Book note: LLMs are **both** — **prefill** tends compute-bound, **decode** tends memory-bound (Ch. 15–18).

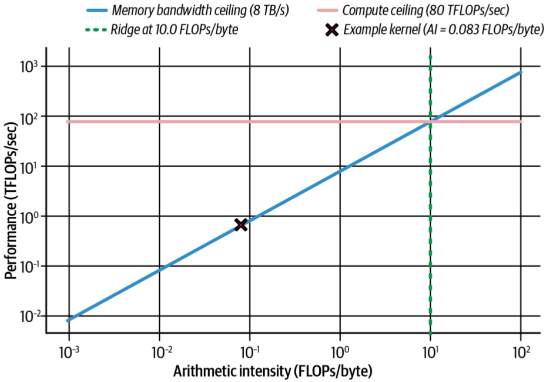


Figure 6-21. Roofline for a Blackwell-class GPU; our kernel sits at 0.083 FLOPs/byte

Plain English: the roofline asks first: is this kernel ***starving for math, or for data****?*[4]

Moving Right on the Roofline: Lower Precision Pays Twice

- To escape the memory-bound region, **do more work per byte**: reuse data on-chip, fuse ops — or simply **shrink the bytes**.
- One 128-byte transaction carries **32 FP32 = 64 FP16 = 128 FP8 = 256 FP4** values. FP32 $\rightarrow$ FP16 instantly **doubles** FLOPs/byte; FP8 doubles throughput and halves memory again vs. FP16.
- Blackwell adds native **FP8 and FP4 Tensor Cores**, plus **hardware decompression**: weights stored compressed in HBM (even beyond FP4 — 4-bit/2-bit schemes) are expanded on the fly and cast to FP16/FP32 for accurate accumulation.
- Net: **effectively more memory bandwidth** — an architectural edge for memory-bound **token generation**.

The takeaway

Precision is a **bandwidth** optimization as much as a compute one: halve the bytes $\Rightarrow$ double the arithmetic intensity $\Rightarrow$ move toward the compute roof.

Profiling Workflow: Diagnose, Fix, Re-measure

- Memory-bound signature in **ncu**: **high DRAM utilization + low ALU utilization** — warps stalled on memory. Also watch **global load efficiency** dropping.
- **nsys** timeline shows idle stretches between kernels = GPU waiting on transfers.
- Expected overlap but see sequential copy-then-kernel? Two usual suspects: an unwanted **default-stream sync**, or a **missing pinned-memory buffer** — without pinned memory, `cudaMemcpyAsync` **cannot overlap** with kernels.
- Newer ncu niceties: **range replay**, source correlation, launch-stack size metric.
- After fixes: idle gaps vanish, **memory pipe utilization** climbs toward peak.

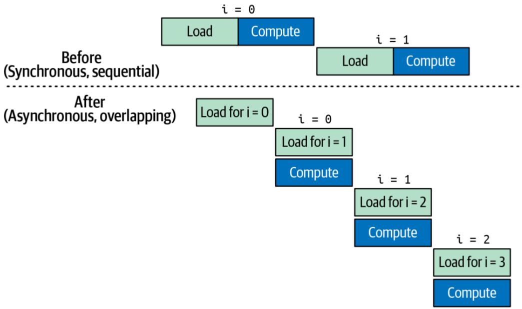


Figure 6-22. Sync (sequential) vs. async (overlapped) transfers + compute *Always measure after each change — profilers confirm whether stalls actually dropped.*

Key Takeaways (the Book's Eight)

- **SIMT**: 32-thread warps in lockstep; many warps in flight = latency hidden.
- **Hierarchy**: threads → blocks (≤1,024) → grids; minimize `__syncthreads()` barriers.
- **Occupancy vs. limits**: multiples of 32; per-SM: 64 warps, 32 blocks, 228 KB shared, 255 regs/thread.
- **Launch params**: 256 threads/block to start; grid = ceil(N/256); tune via profiling.
- **Async memory**: `cudaMallocAsync` + streams + pools; PyTorch's caching allocator ditto.
- **Memory ladder**: reuse in registers/shared/L1; coalesce into L2/HBM.
- **Unified Memory**: prefetch + advise to kill surprise stalls.
- **Roofline**: FLOPs/byte picks your battle; FP16/FP8/FP4 + hardware decompression move you right; **TMEM + UMMA** can shift kernels memory- → compute-bound.

Conclusion and What's Next

This chapter in one sentence

Make the GPU **busy** (occupancy), make data **close** (memory hierarchy), and let the **roofline** tell you which battle to fight next.

- The book's closing caveat: **max occupancy is not always optimal** — with enough **ILP** per thread, moderate/low occupancy can win; sometimes **fewer threads with more registers each** is faster. **Always benchmark.**
- Coming next in the book: warp divergence & memory-access tuning (Ch. 7–8), arithmetic intensity (Ch. 9), TMA & warp specialization (Ch. 10).
- My second talk — **Chapter 12** — builds on this: once single kernels are fast, how do we **orchestrate** them so the GPU never waits for the CPU?

Questions?

References & Further Reading

1. C. Fregly. *AI Systems Performance Engineering* (O'Reilly, 2026), Chapter 6.
2. NVIDIA. *Blackwell Tuning Guide* and *Blackwell Architecture Whitepaper* — SM limits, dual-issue schedulers, TMEM/tcgen05.
3. NVIDIA. *CUDA C++ Programming Guide* — thread hierarchy, Unified Memory, occupancy API, `__launch_bounds__`, stream-ordered allocation, PTX/fatbin.
4. S. Williams, A. Waterman, D. Patterson. *Roofline: An Insightful Visual Performance Model for Multicore Architectures*. CACM 52(4), 2009.
5. NVIDIA. *Compute Sanitizer User Guide* — memcheck, racecheck, initcheck, synccheck.
6. NVIDIA. *Nsight Systems* and *Nsight Compute* documentation — `nsys profile`, `ncu --section SpeedOfLight`, NVTX.
7. Book code and benchmarks: the book's GitHub repository (per-architecture results for the illustrative tables in this deck).